8
Backtracking

Modern computers are so fast that brute force can be an effective and honorable way to solve problems. For example, sometimes it is easier to count the number of items in a set by actually constructing them than by using sophisticated combinatorial arguments. Of course, this requires the number of items searched to be small enough for the computation to complete.

A modern personal computer has a clock rate of about 1 gigahertz, meaning one billion operations per second. Figure that doing anything interesting takes a few hundred instructions or even more. Thus you can hope to search a few million items per second on contemporary machines.

It is important to realize how big (or how small) one million is. One million permutations means all arrangements of roughly 10 or 11 objects, but not more. One million subsets means all combinations of roughly 20 items, but not more. Solving significantly larger problems requires carefully pruning the search space to make sure we look at only the elements which really matter.

In this chapter, we look at backtracking algorithms for exhaustive search and designing effective pruning techniques to make them as powerful as possible.

8.1 Backtracking

Backtracking is a systematic method to iterate through all the possible configurations of a search space. It is a general algorithm/technique which must be customized for each individual application.

In the general case, we will model our solution as a vector $a = (a_1, a_2, ..., a_n)$, where each element a_i is selected from a finite ordered set S_i. Such a vector might represent an arrangement where a_i contains the ith element of the permutation. Or the vector might represent a given subset S, where a_i is true if and only if the ith element of the universe is in S. The vector can even represent a sequence of moves in a game or a path in a graph, where a_i contains the ith event in the sequence.

At each step in the backtracking algorithm, we start from a given partial solution, say, $a = (a_1, a_2, ..., a_k)$, and try to extend it by adding another element at the end. After extending it, we must test whether what we have so far is a solution – if so, we should print it, count it, or do what we want with it. If not, we must then check whether the partial solution is still potentially extendible to some complete solution. If so, recur and continue. If not, we delete the last element from a and try another possibility for that position, if one exists.

The honest working code is given below. We include a global `finished` flag to allow for premature termination, which could be set in any application-specific routine.

```
bool finished = FALSE;                /* found all solutions yet? */

backtrack(int a[], int k, data input)
{
        int c[MAXCANDIDATES];         /* candidates for next position */
        int ncandidates;              /* next position candidate count */
        int i;                        /* counter */

        if (is_a_solution(a,k,input))
                process_solution(a,k,input);
        else {
                k = k+1;
                construct_candidates(a,k,input,c,&ncandidates);
                for (i=0; i<ncandidates; i++) {
                        a[k] = c[i];
                        backtrack(a,k,input);
                        if (finished) return;   /* terminate early */
                }
        }
}
```

The application-specific parts of this algorithm consists of three subroutines:

- `is_a_solution(a,k,input)` — This Boolean function tests whether the first k elements of vector a are a complete solution for the given problem. The last argument, `input`, allows us to pass general information into the routine. We will use it to specify n, the size of a target solution. This makes sense when constructing permutations of size n or subsets of n elements, but may not be relevant when constructing variable-sized objects such as sequences of moves in a game. In such applications, this last argument can be ignored.

- `construct_candidates(a,k,input,c,ncandidates)` — This routine fills an array c with the complete set of possible candidates for the kth position of a, given the contents of the first $k-1$ positions. The number of candidates returned in this array is denoted by `ncandidates`. Again, `input` may be used to pass auxiliary information, particularly the desired solution-size.
- `process_solution(a,k)` — This routine prints, counts, or somehow processes a complete solution once it is constructed. Note that the auxiliary `input` is unnecessary here, since k denotes the number of elements in the solution.

Backtracking ensures correctness by enumerating all possibilities. It ensures efficiency by never visiting a state more than once.

Study how recursion yields an elegant and easy implementation of the backtracking algorithm. Because a new candidates array c is allocated with each recursive procedure call, the subsets of not-yet-considered extension candidates at each position will not interfere with each other. We will see that depth-first traversal in graphs (Chapter 9) uses essentially the same recursive algorithm as `backtrack`. Backtracking can be thought of as depth-first search over an implicit graph instead of an explicit one.

We now provide two examples of backtracking in action, by giving specific implementations of these three functions which iterate through all subsets and permutations of n elements.

8.2 Constructing All Subsets

As mentioned above, we can construct the 2^n subsets of n items by iterating through all possible 2^n length-n vectors of *true* or *false*, letting the ith element denote whether item i is or is not in the subset.

Using the notation of the general backtrack algorithm, $S_k = (true, false)$, and a is a solution whenever $k \geq n$. We can now construct all subsets with simple implementations for `is_a_solution()`, `construct_candidates()`, and `process_solution()`. Printing each subset out after constructing it actually proves to be the most complicated of the three routines!

```
is_a_solution(int a[], int k, int n)
{
        return (k == n);                /* is k == n? */
}

construct_candidates(int a[], int k, int n, int c[], int *ncandidates)
{
        c[0] = TRUE;
        c[1] = FALSE;
        *ncandidates = 2;
}
```

```
process_solution(int a[], int k)
{
        int i;                          /* counter */

        printf("{");
        for (i=1; i<=k; i++)
                if (a[i] == TRUE) printf(" %d",i);

        printf(" }\n");
}
```

Finally, we must instantiate the call to `backtrack` with the right arguments. Specifically, this means giving a pointer to the empty solution vector, setting $k = 0$ to denote that it is empty, and specifying the number of elements in the universal set:

```
generate_subsets(int n)
{
        int a[NMAX];                    /* solution vector */

        backtrack(a,0,n);
}
```

In what order will this routine generate the subsets of $\{1, 2, 3\}$? The critical issue is the order of moves generated in `construct_candidates`. Because *true* always appears before *false*, the subset of all trues is generated first, and the all-false empty set is generated last:

```
{ 1 2 3 }
{ 1 2 }
{ 1 3 }
{ 1 }
{ 2 3 }
{ 2 }
{ 3 }
{ }
```

8.3 Constructing All Permutations

The problem of constructing all permutations is similar to that of generating all subsets, except that the candidates for the next move now depend on the values in the partial solution. To avoid repeating permutation elements, we must ensure that the ith element of the permutation is distinct from all the elements before it.

To use the notation of the general backtrack algorithm, $S_k = \{1, \ldots, n\} - a$, and a is a solution whenever $k = n$:

```
construct_candidates(int a[], int k, int n, int c[], int *ncandidates)
{
        int i;                          /* counter */
        bool in_perm[NMAX];             /* who is in the permutation? */

        for (i=1; i<NMAX; i++) in_perm[i] = FALSE;
        for (i=0; i<k; i++) in_perm[ a[i] ] = TRUE;

        *ncandidates = 0;
        for (i=1; i<=n; i++)
                if (in_perm[i] == FALSE) {
                        c[ *ncandidates] = i;
                        *ncandidates = *ncandidates + 1;
                }
}
```

Testing whether i is a candidate for the kth slot in the permutation can be done by iterating through all $k-1$ elements of a and verifying that none of them matched, but we prefer to set up a bit-vector data structure (see Chapter 2) to maintain which elements are in the partial solution. This gives a constant-time legality check.

Completing the job of generating permutations requires specifying `process_solution` and `is_a_solution`, as well as setting the appropriate arguments to `backtrack`. All are essentially the same as for subsets:

```
process_solution(int a[], int k)
{
        int i;                          /* counter */

        for (i=1; i<=k; i++) printf(" %d",a[i]);

        printf("\n");
}

is_a_solution(int a[], int k, int n)
{
        return (k == n);
}

generate_permutations(int n)
{
        int a[NMAX];                    /* solution vector */

        backtrack(a,0,n);
}
```

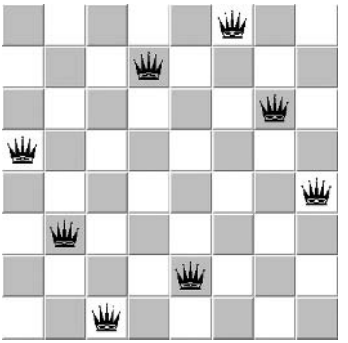


Figure 8.1. A solution to the eight-queens problem.

Note that these routines generate the permutations in *lexicographic*, or sorted order, i.e., `123`, `132`, `213`, `231`, `312`, and `321`.

8.4 Program Design Example: The Eight-Queens Problem

The eight queens problem is a classical puzzle of positioning eight queens on an 8×8 chessboard such that no two queens threaten each other. This means that no two queens may lie on the same row, column or diagonal, as shown in Figure 8.1. It has been studied by many famous mathematicians over the years, including Gauss, as well as countless not-so-famous ones who have taken basic programming courses.

There is nothing in this problem which prevents us from considering larger values than eight. The n-queens problem asks how many distinct ways there are to place n mutually non-attacking queens on an $n \times n$ chessboard. For even modest-sized n, there are far too many solutions to make it interesting to print them out. But to what size n can we count them all in a modest amount of time?

Solution starts below

Getting a grip on such a problem usually requires trying to construct small solutions by hand. You should be able to convince yourself that there is no solution for $n = 2$, since the second queen must threaten the first along a row, column, or diagonal. There is more room to maneuver for $n = 3$, but trial and error should convince you there is still no solution. We encourage you to try to construct a solution for $n = 4$, the smallest interesting case.

Implementing a `backtrack` search requires us to think carefully about the most concise, efficient way to represent our solutions as a vector. What is a reasonable representation for an n-queens solution, and how big must it be?

The most straightforward representation might be to emulate our subset generator, and use a solution vector where a_i is true if and only if there is a queen on the ith square. This requires giving each square a unique name from 1 to n^2. The set of candidate solutions for the ith square will be *true* if none of the previously placed queens threaten

this square, and *false* otherwise. We have a solution after all n^2 squares have values where exactly n of them are *true*.

Is this a good representation? It doesn't seem very concise, since almost all the elements end up with *false* in a successful solution. This means it is also very expensive. There are $2^{64} \approx 1.84 \times 10^{19}$ different true/false vectors for an 8×8 board, and while not all of them will be completely constructed it is a frightening number to contemplate.

What about having the ith element of the solution vector explicitly list the square where the ith queen resides? In this representation, a_i will be an integer from 1 to n^2, giving us a solution whenever we have successfully filled in the first n elements of a. The candidates for the ith position are all the squares which are not threatened by any of the first $i-1$ queens.

Is this better than the previous solution? There are "only" $64^8 \approx 2.81 \times 10^{14}$ possible vectors with this representation for an 8×8 board. This is a huge improvement, but still far above the order-of-one million (10^6) search space that starts to restrict feasibility. Making backtracking work requires eliminating, or *pruning*, the vast majority of these possibilities before they are constructed.

8.5 Pruning Search

The combinatorial explosion implies that the size of most search spaces grows exponentially with the size of the problem. Thus even modest-sized problems quickly hit a wall beyond which they cannot continue to make progress in reasonable amounts of time. To make a backtracking program efficient enough to solve interesting problems, we must prune the search space by terminating every search path the instant it becomes clear it cannot lead to a solution.

The term *prune* for this operation is particularly appropriate. The gardener prunes his trees by cutting back dead and misshapen branches so the tree can focus its energies productively. Similarly, the recursive calls of `backtrack` define a tree. Pruning this tree, by establishing that the set of candidate extensions at a position is in fact empty, keeps it from growing uncontrollably.

So how can we prune search with the positional representation described above? First, we can remove symmetries. As thus far stated, there is no difference between the queen in the first position of the vector (a_1) and the queen in the second position (a_2). Left unchecked, this will generate each solution $8! = 40{,}320$ times! This can be easily corrected by ensuring that the queen in a_i sits on a higher number square than the queen in a_{i-1}. This simple change will reduce the search space to $\binom{64}{8} = 4.426 \times 10^9$.

Thinking more deeply about the problem leads to an even better representation. Note that there must be exactly one queen per row for an n-queens solution. Why? If there were zero queens in one row, then there must be at least two queens in some other row to get a total of n queens. But two queens in the same row are impossible, because they threaten each other. Limiting the candidates of the ith queen to the eight squares on the ith row reduces our search space to $8^8 \approx 1.677 \times 10^7$, a large but manageable size.

But we can do even better! Since no two queens can occupy the same column, we know that the n columns of a complete solution must form a permutation of n. By avoiding repetitive elements, we reduce our search space to just $8! = 40{,}320$ – clearly short work for any reasonably fast machine.

Now we are ready to code up our solution. The critical routine is the candidate constructor. We repeatedly check whether the kth square on the given row is threatened by any previously positioned queen. If so, we move on, but if not we include it as a possible candidate:

```
construct_candidates(int a[], int k, int n, int c[], int *ncandidates)
{
      int i,j;                          /* counters */
      bool legal_move;                  /* might the move be legal? */

      *ncandidates = 0;
      for (i=1; i<=n; i++) {
          legal_move = TRUE;
          for (j=1; j<k; j++) {
                  if (abs((k)-j) == abs(i-a[j]))  /* diagonal threat */
                          legal_move = FALSE;
                  if (i == a[j])                  /* column threat */
                          legal_move = FALSE;
          }
          if (legal_move == TRUE) {
                  c[*ncandidates] = i;
                  *ncandidates = *ncandidates + 1;
          }
      }
}
```

The remaining routines are simple, particularly since we are only interested in counting the solutions, not displaying them:

```
process_solution(int a[], int k)
{
        int i;                            /* counter */

        solution_count ++;
}

is_a_solution(int a[], int k, int n)
{
        return (k == n);
}
```

```
nqueens(int n)
{
        int a[NMAX];                    /* solution vector */

        solution_count = 0;
        backtrack(a,0,n);
        printf("n=%d  solution_count=%d\n",n,solution_count);
}
```

The modest laptop computer we wrote this on solved up to $n = 9$ instantly. Beyond that, it started laboring. The fan on the laptop turned on around $n = 10$ because the computation started generating enough heat to require cooling. By $n = 14$ it was taking several minutes, enough for us to lose interest in larger n. Besides, the fan was giving us a headache. Our counts:

```
n=1  solution_count=1
n=2  solution_count=0
n=3  solution_count=0
n=4  solution_count=2
n=5  solution_count=10
n=6  solution_count=4
n=7  solution_count=40
n=8  solution_count=92
n=9  solution_count=352
n=10  solution_count=724
n=11  solution_count=2680
n=12  solution_count=14200
n=13  solution_count=73712
n=14  solution_count=365596
```

More efficient programs could certainly go a little farther. Our candidate generator can be made faster by terminating the inner for-loop soon as the Boolean variable turns false. More time might be saved by additional pruning. In the current implementation, we backtrack as soon as the kth row has no legal moves. But if some subsequent row (say, the $(k + 2)$nd) has no legal moves, anything we do on the kth row is ultimately futile. The sooner we figure this out, the better.

More profitably, we can try to exploit symmetry. Rotating any solution by 90 degrees yields a different solution, as does reflecting it through the center of the board. By carefully generating only one solution in each equivalence class and counting the symmetries, we can substantially reduce the amount of search needed.

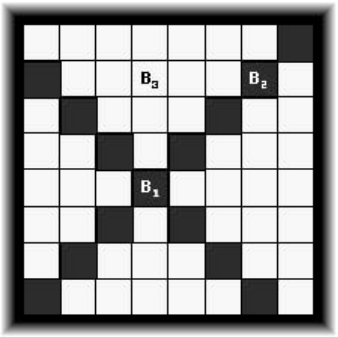
It is fun to try to make a search program as efficient as possible. Why don't you give the queens a try and see how high a value of n you can search within one minute? Don't expect to get *too* much farther than we did, because solving to $n + 1$ should require about ten times as much computation as getting to n in this size range. Even very small increases in solvable problem size thus represents a substantial victory.

8.6 Problems

8.6.1 Little Bishops

PC/UVa IDs: 110801/861, **Popularity:** C, **Success rate:** high **Level:** 2

A bishop is a piece used in the game of chess which can only move diagonally from its current position. Two bishops attack each other if one is on the path of the other. In the figure below, the dark squares represent the reachable locations for bishop B_1 from its current position. Bishops B_1 and B_2 are in attacking position, while B_1 and B_3 are not. Bishops B_2 and B_3 are also in non-attacking position.



Given two numbers n and k, determine the number of ways one can put k bishops on an $n \times n$ chessboard so that no two of them are in attacking positions.

Input

The input file may contain multiple test cases. Each test case occupies a single line in the input file and contains two integers $n(1 \leq n \leq 8)$ and $k(0 \leq k \leq n^2)$.

A test case containing two zeros terminates the input.

Output

For each test case, print a line containing the total number of ways one can put the given number of bishops on a chessboard of the given size so that no two of them lie in attacking positions. You may safely assume that this number will be less than 10^{15}.

Sample Input	*Sample Output*
8 6 4 4 0 0	5599888 260

8.6.2 15-Puzzle Problem

PC/UVa IDs: 110802/10181, **Popularity:** B, **Success rate:** average **Level:** 3

The 15-puzzle is a very popular game: you have certainly seen it even if you don't know it by that name. It is constructed with 15 sliding tiles, each with a different number from 1 to 15, with all tiles packed into a 4 by 4 frame with one tile missing. The object of the puzzle is to arrange the tiles so that they are ordered as below:



The only legal operation is to exchange the missing tile with one of the 2, 3, or 4 tiles it shares an edge with. Consider the following sequence of moves:



A random puzzle position



The missing tile moves right (`R`)



The missing tile moves upwards (`U`)



The missing tile moves left (`L`)

We denote moves by the neighbor of the missing tile is swapped with it. Legal values are "`R`," "`L`," "`U`," and "`D`" for right, left, up, and down, based on the movements of the hole.

Given an initial configuration of a 15-puzzle you must determine a sequence of steps that take you to the final state. Each *solvable* 15-puzzle input requires at most 45 steps to be solved with our judge solution; you are limited to using at most 50 steps to solve the puzzle.

Input

The first line of the input contains an integer n indicating the number of puzzle set inputs. The next $4n$ lines contain n puzzles at four lines per puzzle. Zero denotes the missing tile.

Output

For each input set you must produce one line of output. If the given initial configuration is not solvable, print the line "`This puzzle is not solvable.`" If the puzzle is solvable, then print the move sequence as described above to solve the puzzle.

Sample Input

```
2
2 3 4 0
1 5 7 8
9 6 10 12
13 14 11 15
13 1 2 4
5 0 3 7
9 6 10 12
15 8 11 14
```

Sample Output

```
LLLDRDRDR
This puzzle is not solvable.
```

8.6.3 Queue

PC/UVa IDs: 110803/10128, **Popularity:** B, **Success rate:** high **Level:** 2

Consider a queue with N people, each of a different height. A person can see out to the left of the queue if he or she is taller than all the people to the left; otherwise the view is blocked. Similarly, a person can see to the right if he or she is taller than all the people to the right.

A crime has been committed, where a person to the left of the queue has killed a person to the right of the queue using a boomerang. Exactly P members of the queue had unblocked vision to the left and and exactly R members had unblocked vision to the right, thus serving as potential witnesses.

The defense has retained you to determine how many permutations of N people have this property for a given P and R.

Input

The input consists of T test cases, with T ($1 \leq T \leq 10{,}000$) given on the first line of the input file.

Each test case consists of a line containing three integers. The first integer N indicates the number of people in a queue ($1 \leq N \leq 13$). The second integer corresponds to the number of people who have unblocked vision to their left (P). The third integer corresponds to the number of people who have unblocked vision to their right (R).

Output

For each test case, print the number of permutations of N people where P people can see out to the left and R people can see out to the right.

Sample Input

```
3
10 4 4
11 3 1
3 1 2
```

Sample Output

```
90720
1026576
1
```

8.6.4 Servicing Stations

PC/UVa IDs: 110804/10160, **Popularity:** B, **Success rate:** low **Level:** 3

A company offers personal computers for sale in N towns ($3 \leq N \leq 35$), denoted by $1, 2, \ldots, N$. There are direct routes connecting M pairs among these towns. The company decides to build servicing stations to ensure that for any town X, there will be a station located either in X or in some immediately neighboring town of X.

Write a program to find the minimum number of stations the company has to build.

Input

The input consists of multiple problem descriptions. Every description starts with number of towns N and number of town-pairs M, separated by a space. Each of the next M lines contains a pair of integers representing connected towns, at one pair per line with each pair separated by a space. The input ends with $N = 0$ and $M = 0$.

Output

For each input case, print a line reporting the minimum number of servicing stations needed.

Sample Input

```
8 12
1 2
1 6
1 8
2 3
2 6
3 4
3 5
4 5
4 7
5 6
6 7
6 8
0 0
```

Sample Output

```
2
```

8.6.5 Tug of War

PC/UVa IDs: 110805/10032, **Popularity:** B, **Success rate:** low **Level:** 2

Tug of war is a contest of brute strength, where two teams of people pull in opposite directions on a rope. The team that succeeds in pulling the rope in their direction is declared the winner.

A tug of war is being arranged for the office picnic. The picnickers must be fairly divided into two teams. Every person must be on one team or the other, the number of people on the two teams must not differ by more than one, and the total weight of the people on each team should be as nearly equal as possible.

Input

The input begins with a single positive integer on a line by itself indicating the number of test cases following, each described below and followed by a blank line.

The first line of each case contains n, the number of people at the picnic. Each of the next n lines gives the weight of a person at the picnic, where each weight is an integer between 1 and 450. There are at most 100 people at the picnic.

Finally, there is a blank line between each two consecutive inputs.

Output

For each test case, your output will consist of a single line containing two numbers: the total weight of the people on one team, and the total weight of the people on the other team. If these numbers differ, give the smaller number first.

The output of each two consecutive cases will be separated by a blank line.

Sample Input

```
1

3
100
90
200
```

Sample Output

```
190 200
```

8.6.6 Garden of Eden

PC/UVa IDs: 110806/10001, **Popularity:** B, **Success rate:** average **Level:** 2

Cellular automata are mathematical idealizations of physical systems in which both space and time are discrete, and the physical quantities take on a finite set of discrete values. A cellular automaton consists of a lattice (or array) of discrete-valued variables. The state of such automaton is completely specified by the values of the variables at each position in the lattice. Cellular automata evolve in discrete time steps, with the value at each position (cell) being affected by the values of variables at sites in its neighborhood on the previous time step. For each automaton there is a set of rules that define its evolution.

For most cellular automata there are configurations (states) that are unreachable: no state will produce them by the application of the evolution rules. These states are called Gardens of Eden, because they can only appear as initial states. As an example, consider a trivial set of rules that evolve every cell into 0. For this automaton, any state with non-zero cells is a Garden of Eden.

In general, finding the ancestor of a given state (or the non-existence of such an ancestor) is a very hard, computing-intensive, problem. For the sake of simplicity we will restrict the problem to one-dimensional binary finite cellular automata. In other words, the number of cells is a finite number, the cells are arranged in a linear fashion, and their state will be either "`0`" or "`1`." To simplify the problem further, each cell state will depend only on its previous state and that of its immediate left and right neighbors.

The actual arrangement of the cells will be along a circle, so that the last cell is a neighbor of the first cell.

Problem definition

Given a circular binary cellular automaton, you must determine whether a given state is a Garden of Eden or a reachable state. The cellular automaton will be described in terms of its evolution rules. For example, the table below shows the evolution rules for the automaton: $Cell = XOR(Left, Right)$.

Left $[i-1]$	Cell $[i]$	Right $[i+1]$	New State	
0	0	0	0	$0 * 2^0$
0	0	1	1	$1 * 2^1$
0	1	0	0	$0 * 2^2$
0	1	1	1	$1 * 2^3$
1	0	0	1	$1 * 2^4$
1	0	1	0	$0 * 2^5$
1	1	0	1	$1 * 2^6$
1	1	1	0	$0 * 2^7$
				90 = Automaton Identifier

With the restrictions imposed on this problem, there are only 256 different automata. An identifier for each automaton can be generated by taking the *new state* vector and interpreting it as a binary number, as shown in the table. The example automaton has identifier 90, while the *identity* automaton (where every state evolves to itself) has identifier 204.

Input

The input will consist of several test cases. Each input case describes a cellular automaton and a state on a single line. The first item on the line will be the identifier of the cellular automaton you must work with. The second item in the line will be a positive integer N ($4 \leq N \leq 32$) indicating the number of cells for this test case. Finally, the third item in the line will be a state represented by a string of exactly N zeros and ones. Your program must keep reading lines until the end of file.

Output

If an input case describes a Garden of Eden, output the string `GARDEN OF EDEN`. If the input does not describe a Garden of Eden (it is a reachable state) you must output the string `REACHABLE`.

The output for each test case must be on a different line.

Sample Input

```
0 4 1111
204 5 10101
255 6 000000
154 16 1000000000000000
```

Sample Output

```
GARDEN OF EDEN
REACHABLE
GARDEN OF EDEN
GARDEN OF EDEN
```

8.6.7 Color Hash

PC/UVa IDs: 110807/704, **Popularity:** B, **Success rate:** average **Level:** 3

This puzzle consists of two wheels. Both wheels can rotate clockwise and counterclockwise. They contain 21 colored pieces, 10 of which are rounded triangles and 11 of which are separators. The left panel in Figure 8.2 shows the final puzzle position. Note that to perform a one-step rotation you must turn the wheel until you have advanced a triangle and a separator.

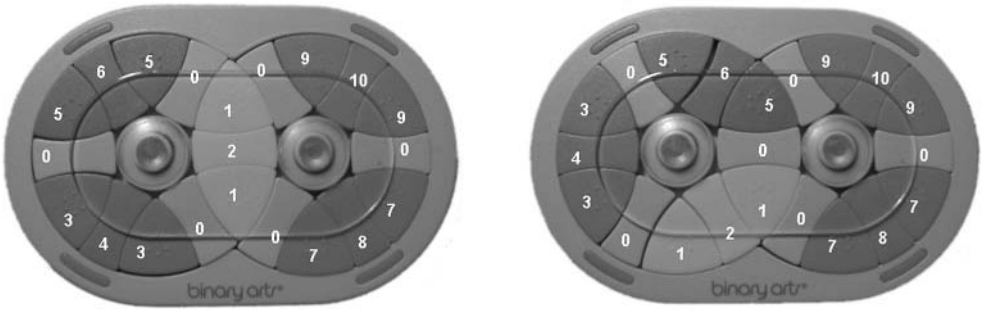


Figure 8.2. Final puzzle configuration (l), with the puzzle after rotating the left wheel on step clockwise from the final configuration (r).

Your job is to write a program that reads the puzzle configuration and prints the minimum sequence of movements required to reach the final position. We will use the following integer values to encode each type of piece:

0	gray separator
1	yellow triangle
2	yellow separator
3	cyan triangle
4	cyan separator
5	violet triangle
6	violet separator
7	green triangle
8	green separator
9	red triangle
10	red separator

A puzzle configuration will be described using 24 integers; the first 12 describe the left wheel configuration; the last 12, the right wheel. The first integer represents the bottom right separator of the left wheel and the next 11 integers describe the left wheel clockwise. The 13th integer represents the bottom left separator of the right wheel and the next 11 integers describe the right wheel counterclockwise.

The final position is therefore encoded

```
0 3 4 3 0 5 6 5 0 1 2 1 0 7 8 7 0 9 10 9 0 1 2 1
```

If we rotate the left wheel clockwise one position from the final configuration (as shown in the right-hand figure) the puzzle configuration would be encoded

```
2 1 0 3 4 3 0 5 6 5 0 1 0 7 8 7 0 9 10 9 0 5 0 1
```

Input

Input for your program consists of several puzzles. The first line of the input will contain an integer n specifying the number of puzzles. There will then be n lines, each containing 24 integers separated with one white space, describing the initial puzzle configuration as explained above.

Output

For each configuration, your program should output one line with just one number representing the solution. Each movement is encoded using one digit from 1 to 4 in the following way:

1 Left Wheel Clockwise rotation
2 Right Wheel Clockwise rotation
3 Left Wheel Counterclockwise rotation
4 Right Wheel Counterclockwise rotation

No space should be printed between each digit. Since multiple solutions could be found, you should print the solution that is encoded as the smallest number. The solution will never require more than 16 movements.

If no solution is found you should print, "`NO SOLUTION WAS FOUND IN 16 STEPS`". If you are given the final position you should print, "`PUZZLE ALREADY SOLVED`".

Sample Input

```
3
0 3 4 3 0 5 6 5 0 1 2 1 0 7 8 7 0 9 10 9 0 1 2 1
0 3 4 5 0 3 6 5 0 1 2 1 0 7 8 7 0 9 10 9 0 1 2 1
0 9 4 3 0 5 6 5 0 1 2 1 0 7 8 7 0 9 10 3 0 1 2 1
```

Sample Output

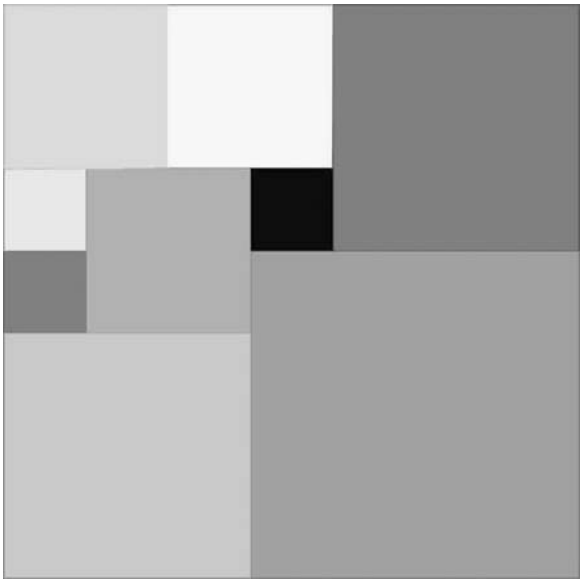
```
PUZZLE ALREADY SOLVED
1434332334332323
NO SOLUTION WAS FOUND IN 16 STEPS
```

8.6.8 Bigger Square Please...

PC/UVa IDs: 110808/10270, **Popularity:** C, **Success rate:** high **Level:** 3

Tomy has many paper squares. The side length (size) of them ranges from 1 to $N-1$, and he has an unlimited number of squares of each kind. But he really wants to have a bigger one – a square of size N.

He can make such a square by building it up from the squares he already has. For example, a square of size 7 can be built from nine smaller squares as shown below:



There should be no empty space in the square, no extra paper outside the square, and the small squares should not overlap. Further, Tomy wants to make his square using the minimal number of possible squares. Can you help?

Input

The first line of the input contains a single integer T indicating the number of test cases. Each test case consists of a single integer N, where $2 \leq N \leq 50$.

Output

For each test case, print a line containing a single integer K indicating the minimal number of squares needed to build the target square. On the following K lines, print three integers x, y, l indicating the coordinates of top-left corner ($1 \leq x, y \leq N$) and the side length of the corresponding square.

Sample Input

```
3
4
3
7
```

Sample Output

```
4
1 1 2
1 3 2
3 1 2
3 3 2
6
1 1 2
1 3 1
2 3 1
3 1 1
3 2 1
3 3 1
9
1 1 2
1 3 2
3 1 1
4 1 1
3 2 2
5 1 3
4 4 4
1 5 3
3 4 1
```

8.7 Hints

8.1 In what ways do we have to modify our eight-queens solution to solve the bishop's problem? Does separating the problem of positioning white bishops from that of black bishops help?

8.2 How can we prevent repeating any puzzle state in the course of our search, which is both inefficient and might lead to excessively long move sequences?

8.3 How can we represent the solution for efficient search? Are we better off constructing permutations, or identifying the subsets of the people with unblocked vision?

8.5 The potential size of this problem makes it a challenge, perhaps too large for backtracking even with sophisticated pruning. Can we keep track of all *team weights* realizable by some subset of the first i people, without explicitly enumerating the 2^i subsets? Note that the number of distinct possible team weights is much smaller than 2^i.

8.7 What is the best way to represent the state of a solution?

8.8 Does it pay to try to position the largest squares first?

8.8 Notes

8.1 There is a nice combinatorial solution to the bishops problem which counts the answers without exhaustive search. This enables us to determine the number of placements for much larger chessboards. UVa judge problem 10237 poses the bishops problem for large enough inputs to make such a solution necessary.

8.4 A set of vertices S in a graph G such that every vertex is either in S or a neighbor of a vertex in S is known as a *dominating set* of the graph. This problem of finding a minimum dominating set is NP-complete, so exhausive search is the only available algorithm if you demand an optimal solution.

8.6 Cellular automata have been proposed as models for a wide variety of natural phenomenon. Read Wolfram's controversal book *A New Kind of Science* [Wol02] to see what all the fuss is about.

8.7 Binary Arts Corp., the creater of Color Hash and many other great puzzles of combinatorial interest, also runs the website *www.puzzles.com*. Check them out.